

How can we detect cheating effectively without compromising system security through kernel-level access?



Gilles MEUNIER

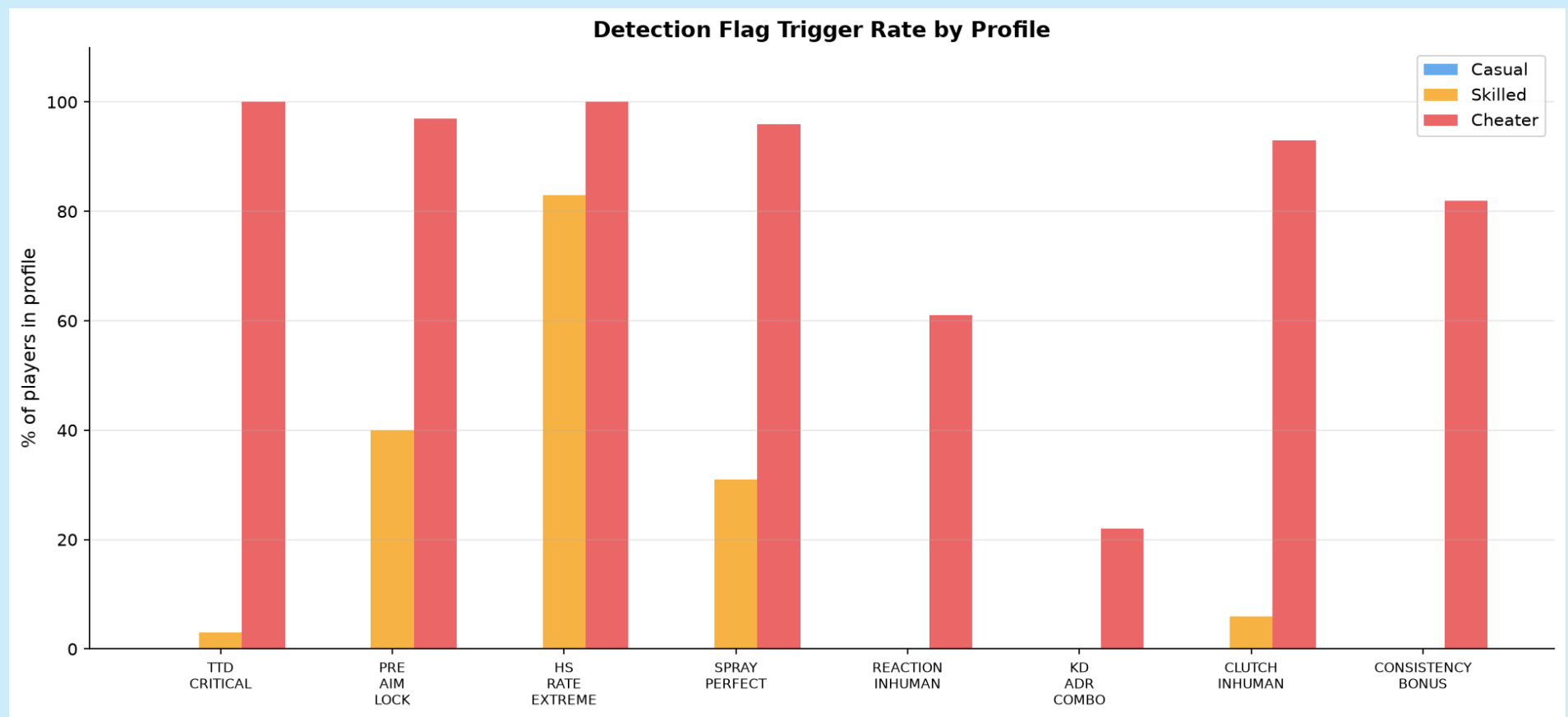
Problem

Kernel-level anti-cheats (Ring 0) are becoming the industry standard for stopping cheaters in competitive FPS games such as Counter-Strike 2, but they require granting a driver unrestricted access to the entire system, not just the game. This creates three risks:

- **Security:** expands the attack surface
- **Privacy:** violates GDPR principles of minimization data access
- **Trust:** drives boycotts, excludes Linux users

This thesis explores an alternative: detecting cheaters through server-side behavioral analysis of gameplay telemetry, without ever touching the player’s machine.

Detection Flags

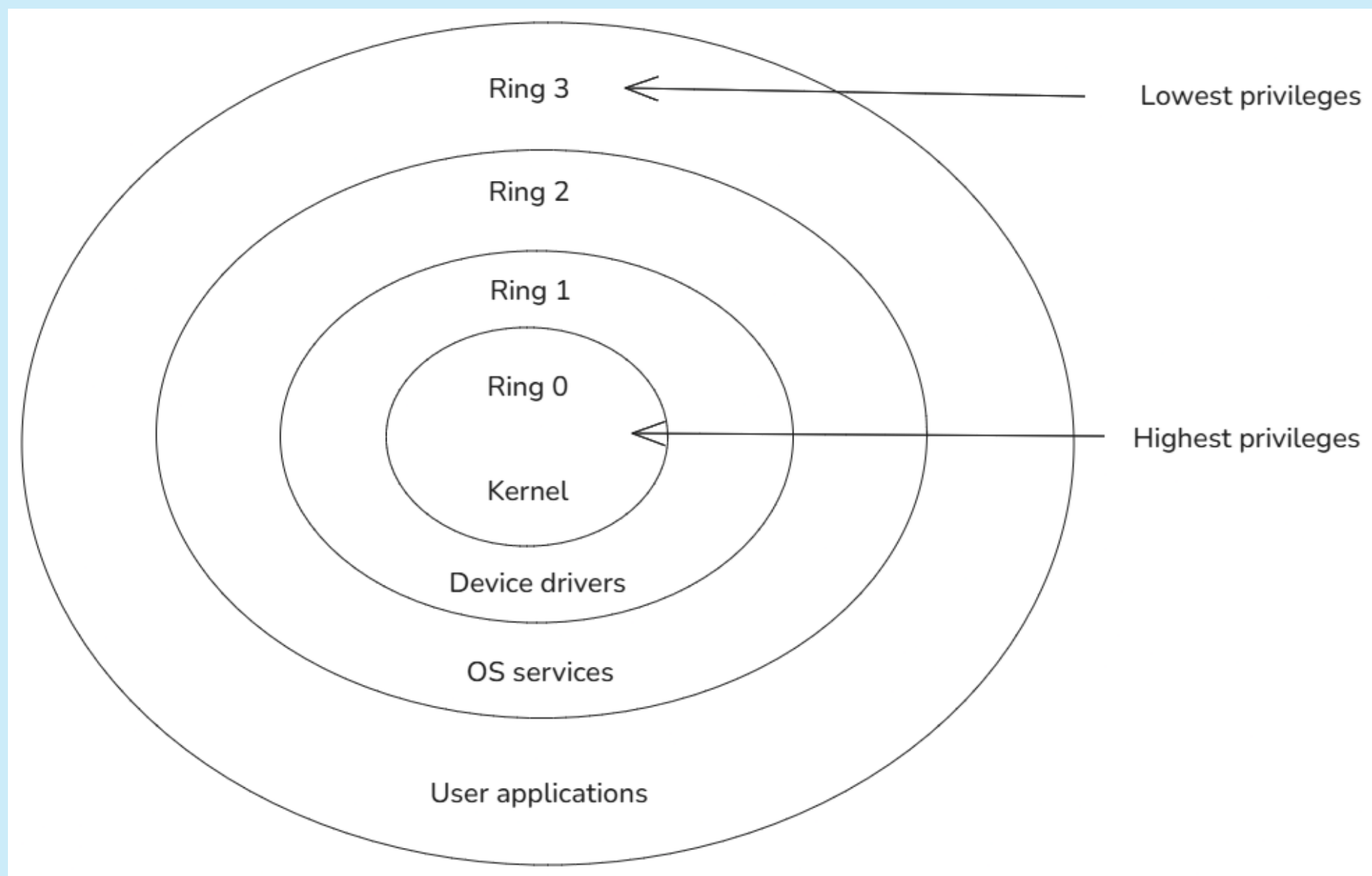


Five of the eight flags fire in over 90% of cheater sessions but stay silent on skilled players — except HS_RATE_EXTREME, which also fires for 83% of skilled sessions, exposing it as the weakest, least discriminative signal on its own. Cross-session persistence across *several* flags, not any single flag, is what separates cheaters from skilled players [Loo2025].

Kernel-Level Risks

- Structural arms race: cheat vendors patch around detections faster than publishers can ship kernel updates
- Categorical blind spot against DMA (hardware) cheats: no software footprint on the host at all
- **Experiment:** using Windows-Kernel-Explorer and Cheat Engine on an isolated VM, a plaintext string was located and reconstructed from an *unrelated* Notepad process’s memory
- Confirms that Ring 0 access is not just theoretically broad, but trivially exercisable against any resident process — browsers, password managers included

Privilege Rings



Kernel-level anti-cheats and kernel-level cheats compete for the same Ring 0 authority, leaving no visibility to user-mode software.

Suspicion Score

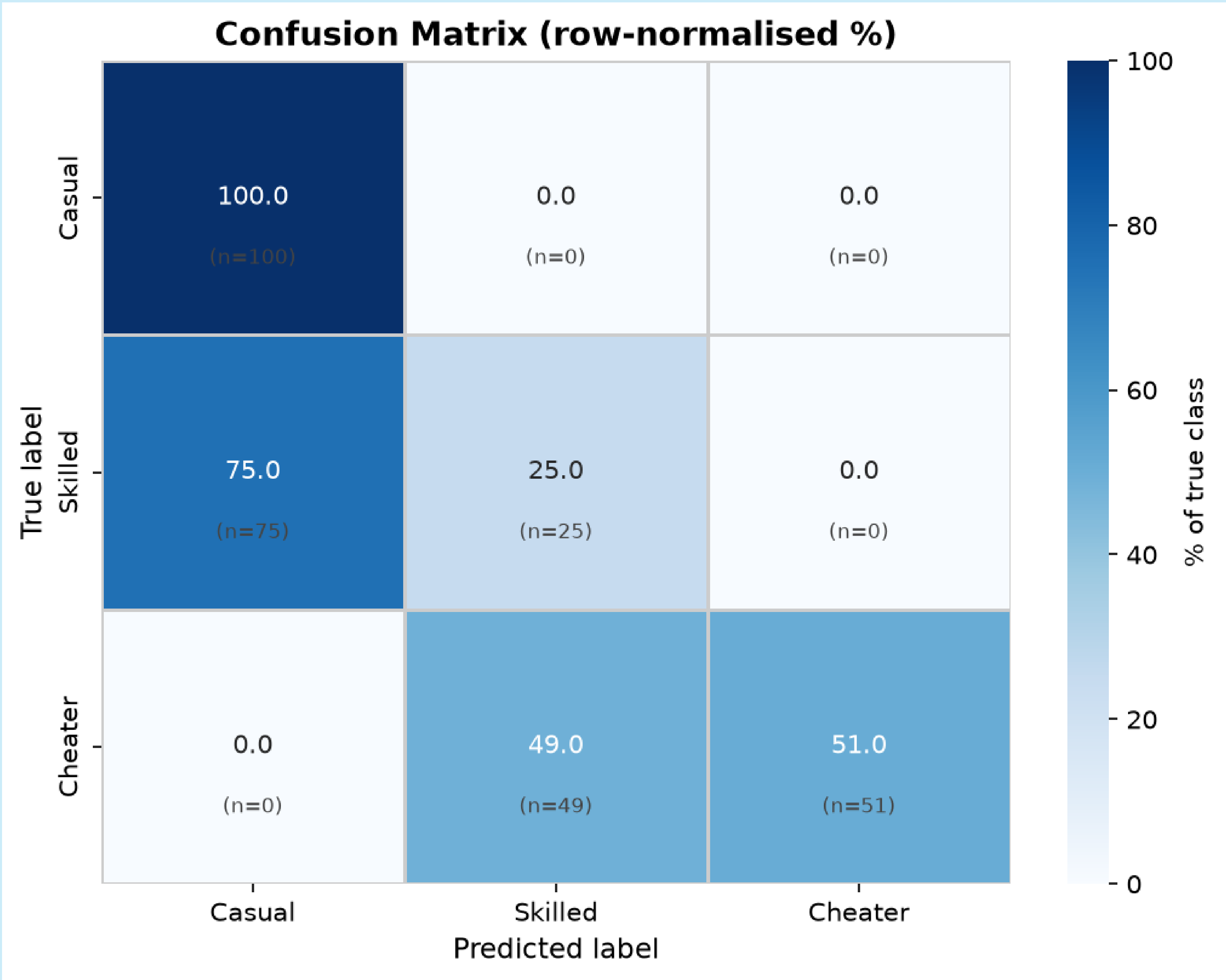
Each player session is scored by two independent layers combined into a single suspicion score:

$$\text{score} = 0.6 \times \text{heuristic} + 0.4 \times (100 \times \text{lstm_prob})$$

The heuristic layer evaluates 8 weighted, threshold-calibrated flags (time-to-damage, pre-aim, spray control, reaction time, ...); the LSTM layer models round-by-round behavioral persistence. A flag only counts if it recurs across a player’s match history, not a single session.

Results

Zero false positives: no legitimate player is ever misclassified as a cheater.



Kernel-level and server-side detection fail on opposite classes of cheat (one blind to hardware-level manipulation, the other blind to a cheat used once and never again), making them complementary layers rather than substitutes [Dorner2024; Collins2024].

References

Acknowledgements

Thanks to my friends for introducing me to Counter-Strike all those years ago, to my teachers for their steady support, and to Hervé Perrachon, my mentor at Saint-Gobain GDII, for sharing his experience throughout this project.